

Programozói dokumentáció

Repülőjegyek

A program felépítése

A program felépítését tekintve 12 modulból tevődik össze, melyek a következők:

- **Check_Search.c** – Ellenőrző és kereső függvények
- **DataStructures.h** – Az adatstruktúrák definiálása
- **FileHandler.c** – Fájlkezelő függvények
- **Foglalas_InputHandler.c** – A felhasználó által megadott foglalások kezelése
- **FoodHandler.c** – A foglalásokhoz tartozó menük kezelése
- **Free_Memory.c** – Memóriaterület felszabadítását végző függvények
- **Jarat_InputHandler.c** – A felhasználó által megadott járatok kezelése
- **JaratokPrint.c** – A járatok megjelenítését végző függvények
- **Logical.c** – Logikai függvények
- **PrintHandler.c** – A menüfelület megjelenítéséért felelős modul
- **SeatHandler.c** – Az ülőhelyeket kezelő modul
- **main.c** – A főprogram

A program a szabványos C könyvtárakon kívül **nem igényel semmilyen más külső könyvtárt** (akár grafikai, akár egyéb). A program fordításához a **GNU GCC** fordítóprogramot ajánlott használni a **Code::Blocks (ver. 20.03)** fejlesztőkörnyezeten belül.

Az adatszerkezetek választása

A programnak négy fő összetevője van adatszerkezetek szempontjából. Ebből az első kettő a **Jarat** és a **Foglalas** elemek, amelyek egyszerű struktúrák. A **Jarat** struktúrában tárolódik egy járat kiindulópontja, úticélja, az utazás ára, a teljes ülésrend sor- és oszlopszáma, valamint a kért menük darabszáma.

A **Foglalas** struktúra hasonló módon épül fel, tehát tartalmazza a foglalást igénybe vevő utas nevét, a lefoglalt járat kiindulópontját és úticélját, valamint a hozzá tartozó árat, a lefoglalt helyeket és az utas által választott menük darabszámát.

```
typedef struct Jarat{           typedef struct Foglalas{
    char honnan[50];           char nev[50];
    char hova[50];             char honnan[50];
    int ar;                    char hova[50];
    int sor;                   int ar;
    int oszlop;                char foglalt_helyek[256][5];
    int kert_menuk[4];         int választott_menuk[4];
}Jarat;                        }Foglalas;
```

Mivel természetesen ezek még nem bővíthető adatszerkezetek, így ebben a formában ezek nem megfelelőek. Ezért a programnak még szüksége van két dinamikus adatszerkezetre is. A **Jaratok** és a **Foglalasok** listák pontosan ezt a célt szolgálják. Mindkettő egy rendezetlen, láncolt lista adatszerkezet, strázsák nélkül. Rendezettséget sem biztosítanak, hiszen a programnak nincsen rá szüksége:

```
typedef struct Jaratok{          typedef struct Foglalasok{
    Jarat elem;                  Foglalas elem;
    struct Jaratok *kov_elem;    struct Foglalasok *kov_fogl;
}Jaratok;                       }Foglalasok;
```

Látható, hogy a listák két elemet tartalmaznak: egy egyszerű struktúrát és a lista következő elemére mutató pointert. Azért erre a formátumra esett a választás, mivel így áláthatóbb ezeknek az adatoknak kezelése. A **Jarat** és a **Foglalas** struktúra már így sok adattagot tartalmaz (névlegesen hatot), és ahelyett, hogy ezek lennének maguk a listák is, inkább szétvannak választva, hogy a listák csak két elemet tartalmazzanak, ezáltal áttekinthetőbbek is.

Nyilvánvalóan ez a fajta megoldás magában hordozza azt is, hogy így egy plusz hivatkozás szükséges minden egyes alkalommal amikor egy listában lévő struktúra egyik elemét szeretnénk elérni. (Pl.: **mozgo->elem.nev**, ahol **mozgo** az adott elemre mutató pointer)

Mindkét listát egy **Jaratok** és egy **Foglalasok** típusú pointer reprezentálja. Az elején mindkét pointer **NULL értéket kap, ez jelenti azt, hogy a listák üresek**.

A program tartalmaz még egy plusz globális, pointerek tömbjét is, ami tartalmazza az egyes menük neveit:

```
extern char *etel[];
```

Az **etel** tömb csupán azt a célt szolgálja, hogy a segítségével egyszerűen megjeleníthetők az ételek nevei, amikor erre szükség van.

A program fontosabb függvényei

Fájlkezelő függvények:

```
Jaratok *Jaratok_read(Jaratok *j_eleje)
```

A függvény feladata beolvasni a **"Jaratok.txt"** fájlból az adatokat és azokat eltárolni a **Jaratok** láncolt listában. Egyetlen paramétere van, a lista elejére mutató pointer. Visszatérési értéke ugyancsak a lista elejére mutató pointer. Ha bármi hiba volt a fájl megnyitásánál, akkor NULL értékkel tér vissza.

```
Foglalasok *Foglalasok_read(Foglalasok *f_eleje)
```

Beolvassa a **"Foglalasok.txt"** fájlban található foglalások adatait és azokat eltárolja a **Foglalasok** listában. Egyetlen paramétere és visszatérési értéke a **Foglalasok** lista elejére mutató pointer. Ha nem sikerült megnyitni a fájlt vagy bármi hiba volt a megnyitása során, akkor a visszatérési értéke NULL.

```
void Jaratok_write(Jaratok *j_eleje, Foglalasok *f_eleje)
```

A függvény feladata, hogy a **Jaratok** lista összes elemét formázva kiírja a **"Jaratok.txt"** fájlba.

```
void Foglalasok_write(Foglalasok *f_eleje)
```

Feladata, hogy a **Foglalasok** lista elemeit formázva kiírja a **"Foglalasok.txt"** fájlba.

```
static void seats_to_write(Jarat elem, Foglalasok *f_eleje, FILE *ptr)
```

Statikus segédfüggvény, az a feladata, hogy az összes olyan ülést megtalálja a paraméterként (2) átadott listában és fájlba írja, amely a paraméterként (1) átadott járáshoz tartozik.

```
static int string_to_int(char *str)
```

Habár nem fájlkezelő függvény, de azoknak a függvényeknek a munkáját segíti. Feladata, hogy a paraméterként kapott sztringet egész számmá alakítsa és azt vissza is adja. Mivelhogy a függvényt csak a fájlkezelő függvények használják, ezért statikus.

A járatokat kezelő függvények:

```
Jarat jarat_megadas(Jaratok *eleje)
```

A függvény a felhasználóval kommunikál és az általa megadott adatokat egy **Jarat** struktúrába menti. A paraméterként átadott listát végig járva ellenőrzi, hogy a felhasználó egy már létező honnan-hova kombinációt adott-e meg. Visszatérési értéke a kész struktúra.

```
bool jarat_exists(Jaratok *eleje, char *hon, char *hov)
```

A függvény ellenőrzi, hogy a paraméterként (1) átadott listában már szerepelnek-e egyszerre a paraméterenként (2 & 3) átadott sztringek az egyes struktúra elemekben. A függvény igaz értékkel tér vissza, ha ez a feltétel teljesül, egyébként pedig hamissal.

```
Jaratok *jarat_beiras(Jaratok *eleje, Jarat uj_jarat)
```

A függvény hozzáfüzi a paraméterként (1) átadott lista végéhez a szintén paraméterként (2) átadott struktúra elemet. Visszatérési értéke az így már új elemet is tartalmazó, a lista elejére mutató pointer.

```
void Jaratok_show_all(Jaratok *j_eleje, Foglalasok *f_eleje, int *idx)
```

Feladata, hogy az egyes járatokat formázva írja ki a képernyőre, amikor erre szükség van. A függvény egyszerre csak 8 db járatot enged ki írni a képernyőre, ha ennél több van, akkor azokat már nem írja ki.

A harmadik paramétereként átadott, egész számra mutató pointerben tárolja a függvény, hogy a **Jaratok** listában meddig kell menni ahhoz, hogy arra az elemre érjünk, amit a felhasználó kiválasztott.

```
void Jaratok_felszabadit(Jaratok *eleje)
```

Felszabadítja a **Jaratok** láncolt listát.

A foglalásokat kezelő függvények:

```
Foglalas fogl_megadas(Jaratok *j_eleje, Foglalasok *f_eleje)
```

A függvény a felhasználó által megadott adatokat eltárolja egy új **Foglalas** struktúrába. Ellenőrzi, hogy a megadott név már létezik-e, továbbá igénybe veszi a **Jaratok_show_all()** függvényt, hogy a felhasználó választani tudjon a járatok közül. Az adatok megadása után visszatér az elkészült **Foglalas** struktúrával.

```
bool name_exists(Foglalasok *eleje, char *name)
```

Ellenőrzi, hogy a paraméterként (1) átadott **Foglalasok** listában már szerepel-e a szintén paraméterként (2) átadott név. A feltétel teljesülése esetén igaz, egyébként hamis értékkel tér vissza.

```
Foglalasok *fogl_beiras(Foglalasok *eleje, Foglalas uj_fogl)
```

A függvény beszúrja a **Foglalasok** lista végére a paraméterként (2) átadott struktúrát, majd visszatér a lista elejére mutató pointerrel.

```
Foglalasok *ules_megadas(Foglalasok *f_eleje, Jaratok *j_eleje)
```

A függvény a név megadása (és ellenőrzése) után megjeleníti az adott foglaláshoz tartozó járat foglaltsági térképét, majd eltárolja a felhasználó által megadott üléseket, miután ellenőrizte, hogy azok már foglaltak-e. Visszatérési értéke a **Foglalasok** lista elejére mutató pointer. Ha a felhasználó egy olyan nevet ad meg, amihez már tartozik olyan foglalás, ahova választottak ülőhelyet, akkor az automatikusan felülíródik az új ülőhely választással.

```
static void fogl_terkep(Foglalasok *f_eleje, Jarat elem)
```

Meg jeleníti az adott járat foglaltsági térképét.

```
bool foglalt_e(Foglalasok *eleje, Foglalas elem, char *ules)
```

Ellenőrzi, hogy a paraméterként (2) átadott ülés már foglalt-e.

```
Foglalasok *etel_megadas(Foglalasok *f_eleje, Jaratok *j_eleje)
```

A függvény a név megadása (és ellenőrzése) után megjeleníti a kiválasztható menüket. A felhasználó által megadott adatokat eltárolja, majd a választás befejeztével visszatér a **Foglalasok** lista elejére mutató pointerrel. Ha egy olyan nevet írt be a felhasználó, amihez már tartozik olyan foglalás, ahol rendeltek ételt, akkor a függvény automatikusan felülírja azt az újjal.

```
static void food_change(Foglalas *f_elem, Jarat *j_elem)
```

Az **etel_megadas()** nevű függvény használja. Feladata, hogy a felhasználó által megadott adatokat eltárolja a paraméterenként átadott struktúrákban (**Pointerként kapja mindkettőt, mivel a függvény megváltoztatja az értéküket**).

```
void Foglalasok_felszabadit(Foglalasok *eleje)
```

Felszabadítja a **Foglalasok** láncolt listáját.

Egy fontos algoritmus

```
int h = strlen(str) - 1;
int i = pow(10, h);
while(str[index] != '\0'){
    r_value += (str[index] - '0') * i;
    i = i / 10;
    index++;
}
```

A fenti kódrészletet a **string_to_int()** függvény tartalmazza:

- Eltárolja a **'h'** egész változóban, a paraméterként átvett (**str**) sztring hosszát.
- Az **'i'** változó eltárolja a **10 h-adik hatványát**. (Ezért vonunk le 1-et a h változóból, mert $10^1 = 10$)
- A ciklust addig kell léptetni, amíg a sztring végére nem értünk.
- A ciklusmagban hozzáadjuk az **r_value** nevű, egész változóhoz a következő karakter **i-szeresét**. (*Char $\rightarrow$ Int konverzió: Az adott karakterből levonjuk a 0-át, mint karaktert.*) **Az utolsó karakter 1-el szorozzuk csak meg.**
- Az **'i'**-t elosztjuk 10-el és az **index** változót növeljük 1-el.

Ezzel a módszerrel visszakapjuk az eredeti számot, amit sztringként kaptunk.